1GIT1 — Fondamentaux de Git

JOUR 1

Contrôle de version avec Git

Historique · Les 3 zones · Commits · Historique · Annulations · .gitignore

📋 CONTENU DE CE DOCUMENT

1. Pourquoi Git ? Origine et philosophie
2. Installation et configuration initiale
3. Créer un dépôt : git init et le dossier .git
4. Les 3 zones fondamentales de Git
5. Observer, indexer, valider : status / add / commit
6. Consulter l'historique : log, diff, HEAD
7. Ignorer des fichiers avec .gitignore
8. Annuler des modifications : restore, revert, reset

1. Pourquoi Git ?

Origine, philosophie et architecture distribuée

Avant de taper la première commande, il est essentiel de comprendre pourquoi Git existe et ce qui le distingue des autres outils de versioning. Git n'est pas simplement un "dossier de sauvegardes" : c'est un système de contrôle de version distribué, conçu dès sa naissance pour être rapide, fiable et décentralisé.

1.1 L'histoire de Git

En 2005, le noyau Linux était développé avec BitKeeper, un outil propriétaire de contrôle de version. Lorsque la licence gratuite de BitKeeper fut révoquée, la communauté Linux se retrouva sans solution adaptée à un projet de cette ampleur. Linus Torvalds, le créateur du noyau Linux, décida alors de créer son propre outil en quelques semaines seulement. Cet outil, il l'appela Git.

ANECDOTE HISTORIQUE

Git a été créé par Linus Torvalds en 2005, suite à la révocation de la licence gratuite de BitKeeper. L'objectif était clair : créer un outil libre, ultra-rapide, et capable de gérer des milliers de contributeurs simultanément sur le noyau Linux.

Aujourd'hui, Git est devenu le standard mondial du versioning, utilisé par des millions de développeurs et de projets open source.

1.2 Contrôle de version : centralisé vs distribué

Il existe deux grandes familles de systèmes de contrôle de version. Comprendre leur différence permet de saisir l'avantage fondamental de Git.

Critère	Centralisé (ex : SVN)	Distribué (Git)
Architecture	Un serveur central unique	**Chaque machine est un dépôt complet**
Travail hors ligne	Impossible (nécessite le réseau)	**Totalement possible**
Historique	Sur le serveur uniquement	**Dupliqué sur chaque poste**
Risque de perte	Si le serveur tombe, tout est perdu	**Chaque clone est une sauvegarde**
Vitesse	Dépend du réseau	**Opérations locales ultra-rapides**

RÉFLEXION

Avec Git, quand vous exécutez git clone, vous ne récupérez pas seulement les fichiers du projet : vous téléchargez l'intégralité de l'historique, toutes les branches, tous les commits. Votre machine locale contient donc une copie complète et autonome du projet.

C'est pourquoi git log, git diff ou git status fonctionnent instantanément, sans connexion réseau.

À TESTER — Mettez en pratique

1. Lancez cette commande et notez la version de Git installée :

```
$ git --version
```

2. Réflexion : si le serveur central tombe en panne dans un système SVN, que se passe-t-il pour les développeurs ? Et avec Git, quelle est la différence ?

2. Installation et Configuration

Préparer son environnement Git

Avant d'utiliser Git, il faut s'assurer qu'il est installé et correctement configuré. La configuration initiale est obligatoire : sans elle, Git refusera d'enregistrer vos commits.

2.1 Vérifier l'installation

Pour vérifier que Git est bien installé et connaître sa version, on utilise la commande suivante :

```
# Vérification de l'installation
$ git --version
git version 2.43.0
```

Si la commande retourne un numéro de version, Git est opérationnel. Dans le cas contraire, rendez-vous sur git-scm.com pour télécharger l'installeur adapté à votre système (Windows, macOS, Linux).

2.2 Configuration obligatoire : identité de l'auteur

Chaque commit Git est signé avec le nom et l'adresse e-mail de son auteur. Ces informations sont stockées directement dans le commit et sont visibles par tous. La configuration est obligatoire : si vous tentez de commiter sans l'avoir faite, Git retournera une erreur.

```
# Configuration de l'identité
$ git config --global user.name "Alice Martin"
$ git config --global user.email "alice.martin@example.com"

# Vérifier la configuration
$ git config --global --list
user.name=Alice Martin
user.email=alice.martin@example.com
```

GLOBAL vs LOCAL

L'option --global applique la configuration à TOUS vos dépôts Git sur cette machine (stockée dans ~/.gitconfig). C'est l'option à utiliser pour votre identité personnelle.

L'option --local (ou sans option) ne s'applique qu'au dépôt courant (stockée dans .git/config). Utile si vous travaillez avec plusieurs identités (perso/pro).

La hiérarchie de priorité : --local > --global > --system

2.3 Configurer l'éditeur par défaut

```
# Configuration de l'éditeur
# Configurer VS Code comme éditeur de commit
$ git config --global core.editor "code --wait"

# Ou Nano pour un éditeur terminal simple
$ git config --global core.editor nano
```

À TESTER — Mettez en pratique

1. Configurez votre identité Git globale avec vos vraies informations :
   ```
   $ git config --global user.name "Prénom Nom"
   $ git config --global user.email "votre@email.com"
   ```
2. Vérifiez que la configuration a bien été enregistrée :
   ```
   $ git config --global --list
   ```
3. Trouvez le fichier .gitconfig sur votre machine. Où se trouve-t-il ?
   Ouvrez-le dans un éditeur pour voir sa structure.

3. Créer un Dépôt Git

git init et la structure interne

Un "dépôt Git" (repository) est un dossier ordinaire dans lequel Git surveille l'évolution des fichiers. Pour transformer un dossier existant en dépôt Git, on utilise une seule commande.

3.1 La commande git init

```
# Initialisation d'un dépôt
# Créer un nouveau dossier et l'initialiser
$ mkdir mon-projet
$ cd mon-projet
$ git init
Initialized empty Git repository in /home/alice/mon-projet/.git/

# Ou initialiser un dossier existant
$ cd projet-existant
$ git init
```

La commande git init ne fait qu'une chose : créer un dossier caché nommé .git à la racine du projet. C'est dans ce dossier que réside toute la "magie" de Git.

3.2 Le dossier .git : le cœur du dépôt

Le dossier .git est la base de données de votre dépôt. Il contient absolument tout ce dont Git a besoin pour fonctionner : l'historique complet, les branches, la configuration locale, les objets compressés, et bien plus encore.

```
mon-projet/
├── src/
│   └── main.py
├── README.md
└── .git/              ← Dossier caché = le dépôt Git
    ├── HEAD            ← Pointeur vers la branche courante
    ├── config          ← Configuration locale du dépôt
    ├── objects/        ← Tous les commits, fichiers, arbres
    ├── refs/           ← Références (branches, tags)
    └── index           ← La Staging Area
```

Structure du dossier .git

RÈGLE ABSOLUE

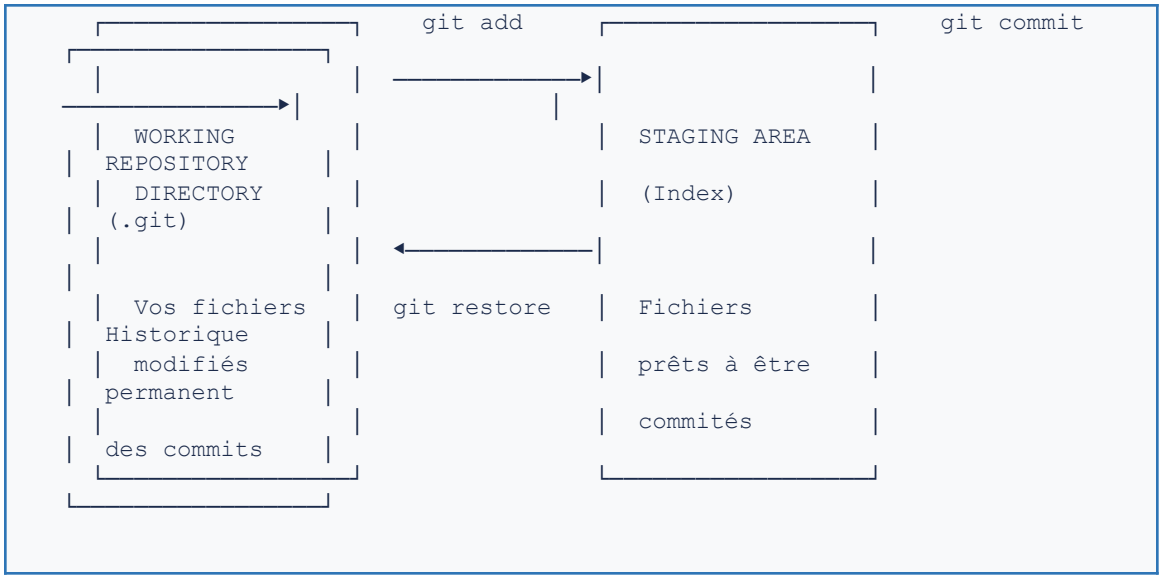
Ne modifiez JAMAIS manuellement le contenu du dossier .git. Ce dossier est géré exclusivement par Git. Toute modification manuelle peut corrompre irrémédiablement votre historique.

Si vous supprimez le dossier .git, vous perdez tout l'historique Git du projet (les fichiers sources restent, mais toute la mémoire de Git disparaît).

4. Les 3 Zones Fondamentales de Git

Working Directory · Staging Area · Repository

C'est le concept le plus important à maîtriser pour travailler efficacement avec Git. Contrairement à d'autres outils qui enregistrent directement les modifications, Git introduit une zone intermédiaire qui vous donne un contrôle fin sur ce que vous voulez historiser.



Le flux de travail Git en 3 zones

4.1 Zone 1 : Working Directory (Répertoire de Travail)

Le Working Directory est tout simplement votre dossier de projet tel que vous le voyez dans votre explorateur de fichiers. C'est là que vous créez, modifiez et supprimez des fichiers. Git surveille ce dossier mais n'enregistre rien automatiquement : il attend vos instructions explicites.

Un fichier dans le Working Directory peut être dans l'un de ces états :

- Untracked (non suivi) : Git ne connaît pas ce fichier, il n'a jamais été ajouté.
- Modified (modifié) : le fichier est connu de Git mais a été changé depuis le dernier commit.
- Staged (indexé) : le fichier a été ajouté à la Staging Area, prêt à être commité.

4.2 Zone 2 : Staging Area (Zone d'Index)

La Staging Area (aussi appelée Index) est une zone de préparation invisible. Elle agit comme un "panier" dans lequel vous déposez précisément les modifications que vous souhaitez inclure dans votre prochain commit. C'est ce qui distingue Git de la plupart des autres systèmes de versioning.

POURQUOI UNE STAGING AREA ?

Imaginez que vous travaillez sur deux fonctionnalités en même temps. Vos fichiers modifiés mélangent les deux. La Staging Area vous permet de sélectionner uniquement les fichiers de la fonctionnalité A pour en faire un commit propre et ciblé, puis de commiter la fonctionnalité B séparément.

Résultat : un historique lisible, où chaque commit correspond à une modification logique et cohérente, plutôt qu'un grand fourre-tout.

C'est un outil de précision, pas une contrainte.

4.3 Zone 3 : Repository (.git)

Le Repository est la base de données permanente de Git, stockée dans le dossier .git. Chaque commit crée un instantané (snapshot) immuable de l'état complet du projet à un moment donné. Ces instantanés forment une chaîne chronologique : l'historique du projet.

Zone	Localisation	Commande pour y entrer	État des fichiers
Working Directory	Votre dossier projet	— (c'est votre espace de travail)	Untracked / Modified
Staging Area	.git/index	**git add <fichier>**	Staged
Repository	.git/objects/	**git commit**	Committed

À TESTER — Mettez en pratique

1. Dans votre dépôt, créez un fichier et observez les 3 états successifs :

```
$ echo 'Bonjour Git' > fichier.txt
$ git status    ← état 1 : Untracked
$ git add fichier.txt
$ git status    ← état 2 : Staged
$ git commit -m "Premier fichier"
$ git status    ← état 3 : rien à commiter (propre)
```

2. Modifiez fichier.txt, puis désindexez-le sans perdre vos modifications :

```
$ echo 'Ligne ajoutée' >> fichier.txt
$ git add fichier.txt
$ git restore --staged fichier.txt
$ git status    ← le fichier doit être 'Modified' mais pas staged
```

5. Observer, Indexer, Valider

git status · git add · git commit

Ces trois commandes forment le cœur du cycle de travail quotidien avec Git. Comprendre ce que chacune fait exactement — et dans quel ordre les utiliser — est la compétence de base du développeur Git.

5.1 git status : observer l'état du projet

git status est la commande que vous utiliserez le plus souvent. Elle affiche une photographie de l'état actuel de votre dépôt : quels fichiers ont été modifiés, quels fichiers sont indexés, quels fichiers sont inconnus de Git.

```
# Exemple de sortie git status
$ git status
On branch main

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
        modified:   src/app.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        tests/test_app.py
```

Message	Signification
Changes to be committed	Fichiers dans la Staging Area, prêts pour le prochain commit

Changes not staged	Fichiers connus de Git, modifiés mais pas encore indexés
Untracked files	Fichiers inconnus de Git, jamais ajoutés au suivi
nothing to commit	Working Directory propre, rien à enregistrer

BONNE PRATIQUE

Exécutez git status AVANT et APRÈS chaque opération Git. C'est votre radar : il vous indique exactement où vous en êtes et ce que Git attend de vous ensuite. Ne naviguez jamais à l'aveugle.

5.2 git add : indexer des modifications

git add déplace des fichiers (ou des portions de fichiers) du Working Directory vers la Staging Area. C'est une étape de sélection consciente : vous choisissez explicitement ce qui entrera dans votre prochain commit.

```
# Utilisation de git add
# Indexer un fichier spécifique
$ git add README.md

# Indexer plusieurs fichiers
$ git add src/app.py tests/test_app.py

# Indexer tous les fichiers modifiés et nouveaux (récursivement)
$ git add .

# Indexer une partie d'un fichier (mode interactif)
$ git add -p src/app.py
```

git add . : avec modération

git add . ajoute TOUT ce qui est modifié ou nouveau dans le dossier courant. C'est pratique, mais dangereux si vous n'avez pas configuré votre .gitignore : vous risquez d'indexer des fichiers secrets (.env, mots de passe), des dossiers de cache (node_modules, __pycache__) ou des fichiers temporaires.

Préférez git add <fichier> pour un contrôle précis, ou configurez .gitignore d'abord.

5.3 git commit : enregistrer un instantané permanent

git commit crée un commit : un instantané permanent et immuable de l'état de la Staging Area à cet instant. Une fois créé, un commit ne peut plus être modifié (seulement annulé par un nouveau commit). Chaque commit reçoit automatiquement son identifiant SHA-1 unique.

```
# Créer un commit
# Commiter avec un message court (forme la plus courante)
$ git commit -m "Ajouter la validation du formulaire de connexion"
[main 3a4b5c6] Ajouter la validation du formulaire de connexion
 2 files changed, 45 insertions(+), 3 deletions(-)

# Commiter avec un message long (ouvre l'éditeur)
$ git commit
```

5.4 Le message de commit : un art à maîtriser

Le message de commit est la trace écrite que vous laissez pour votre futur vous et pour vos collègues. Un bon message répond à une question simple : pourquoi cette modification a-t-elle été faite ?

Mauvais message ❌	Bon message ✅
fix bug	Corriger le crash au démarrage quand la config est absente
update	Mettre à jour les dépendances pour corriger CVE-2024-1234
wip	WIP: Implémenter l'authentification OAuth (en cours)
blah	Refactoriser le module de paiement pour séparer les responsabilités

Dans les projets professionnels, on utilise la convention Conventional Commits : chaque message commence par un type qui qualifie la nature de la modification. C'est un standard adopté par la majorité des projets open source.

Type	Signification (EN)	Traduction (FR)	Exemple de message
feat	New feature	Nouvelle fonctionnalité	feat: add user login form
fix	Bug fix	Correction de bug	fix: crash on empty config file
docs	Documentation only	Documentation uniquement	docs: update README install steps
style	Formatting, no logic	Style/formatage, sans logique	style: reformat code with Black
refactor	Code refactoring	Refactorisation du code	refactor: extract payment module
test	Adding or fixing tests	Ajout ou correction de tests	test: add unit tests for auth
chore	Maintenance, tooling	Maintenance, tâches diverses	chore: update dependencies
perf	Performance improvement	Amélioration des performances	perf: cache DB queries
ci	CI/CD configuration	Intégration continue	ci: add GitHub Actions workflow
build	Build system changes	Système de build	build: upgrade webpack to v5
revert	Reverts a previous commit	Annulation d'un commit précédent	revert: feat: add user login form

FORMAT CONVENTIONAL COMMITS

Format complet : type(scope): description

- type : feat, fix, docs, style... (voir tableau ci-dessus)
- scope : optionnel, précise la zone impactée entre parenthèses
- ! : indique un breaking change (changement majeur incompatible)

Exemples :

feat(auth): add OAuth2 login support

fix(api): handle null response from payment gateway

feat!: remove deprecated /v1 endpoints ← breaking change

Un bon message doit expliquer LE POURQUOI, pas juste le quoi (le quoi est déjà visible dans le diff).

5.5 Le raccourci git commit -am

Pour les fichiers déjà suivis par Git (jamais pour les fichiers untracked), il existe un raccourci qui combine git add et git commit en une seule commande :

```
# Le raccourci -am
# Equivalent à : git add -u && git commit -m "..."
$ git commit -am "Corriger la validation du mot de passe"

# Décomposition des options :
# -a  : indexe automatiquement tous les fichiers DÉJÀ SUIVIS modifiés
# -m  : spécifie le message en ligne
```

LIMITATION IMPORTANTE de -am

git commit -am NE fonctionne PAS pour les nouveaux fichiers (Untracked). L'option -a ne concerne que les fichiers que Git connaît déjà (tracked). Pour les nouveaux fichiers, vous devez toujours passer par git add d'abord.

Exemple : si vous créez un nouveau fichier nouveau.py et que vous essayez git commit -am "...", Git ignorera silencieusement nouveau.py.

À TESTER — Mettez en pratique

1. Faites 3 commits dans votre dépôt (créez ou modifiez des fichiers) :

```
$ echo 'v1' > app.txt && git add app.txt && git commit -m "Version 1"
$ echo 'v2' > app.txt && git commit -am "Version 2"
$ echo 'v3' > app.txt && git commit -am "Version 3"
```

2. Testez le raccourci -am sur un nouveau fichier (untracked) :

```
$ echo 'nouveau' > test.txt
$ git commit -am "Test raccourci"    ← test.txt sera-t-il inclus ?
```

3. Corrigez en ajoutant d'abord avec git add, puis commitez.

4. Question : quelle est la différence entre git commit -m et git commit -am ?

6. Consulter l'Historique

git log · git diff · HEAD et navigation

Git conserve l'intégralité de l'historique de votre projet depuis le premier commit. Savoir naviguer dans cet historique et comparer des états différents du code est une compétence fondamentale.

6.1 git log : explorer l'historique

```
# Sortie de git log
$ git log
commit a3f7c21e8b904d6e5c1234567890abcdef012345 (HEAD -> main)
Author: Alice Martin <alice@example.com>
Date:   Mon Mar 30 09:15:00 2026 +0200

    Ajouter la validation du formulaire de connexion

commit 9b2e4f8a1d7c3e6b5a890123456789cdef012345
Author: Alice Martin <alice@example.com>
Date:   Sun Mar 29 17:30:00 2026 +0200

    Initialiser la structure du projet
```

Par défaut, git log affiche les commits du plus récent au plus ancien. Chaque commit affiche : l'identifiant SHA-1 complet, l'auteur, la date, et le message. Pour un affichage plus compact :

```
# Options utiles de git log
# Vue condensée : SHA-1 abrégé + message sur une ligne
$ git log --oneline
a3f7c21 (HEAD -> main) Ajouter la validation du formulaire de connexion
9b2e4f8 Initialiser la structure du projet

# Avec graphique des branches
$ git log --oneline --graph --all
```

> 💡 **git log --oneline**
>
> L'option --oneline est votre meilleure alliée pour naviguer rapidement dans l'historique. Elle affiche chaque commit sur une seule ligne avec uniquement les 7 premiers caractères du SHA-1 (suffisants pour identifier un commit) et le message.

Ces 7 caractères sont utilisables dans toutes les commandes Git qui acceptent un identifiant de commit : git show a3f7c21, git revert a3f7c21, etc.

6.2 HEAD : le pointeur de position

HEAD est un pointeur spécial qui indique votre position actuelle dans l'historique. En temps normal, HEAD pointe vers la branche courante (ex: main), qui elle-même pointe vers le dernier commit. C'est votre "vous êtes ici" sur la carte de l'historique.

```
Historique Git :

commit-1 ——— commit-2 ——— commit-3 ——— commit-4
(9b2e4f8)    (c1d2e3f)    (7a8b9c0)    (a3f7c21)
                                           ▲
                                         main
                                           ▲
                                         HEAD  ← "vous êtes ici"
```

HEAD pointe vers le commit actuel

La notation HEAD~1 (ou HEAD~) désigne le commit précédant HEAD, HEAD~2 celui d'avant, etc. C'est une façon élégante de naviguer dans l'historique sans avoir à copier-coller des SHA-1.

```
# Notation HEAD~N
# HEAD~1 = le commit parent de HEAD
# HEAD~2 = deux commits en arrière
# HEAD~3 = trois commits en arrière

# Exemples d'utilisation
$ git diff HEAD~1          # Voir ce qui a changé depuis l'avant-dernier
commit
$ git revert HEAD~1        # Annuler l'avant-dernier commit
$ git reset HEAD~2         # Revenir 2 commits en arrière
```

6.3 git diff : comparer des états

git diff est l'outil de comparaison de Git. Selon le contexte, il compare différentes zones ou différents commits.

```
# Les différentes formes de git diff
# Comparer Working Directory vs Staging Area
# (modifications non encore indexées)
$ git diff

# Comparer Staging Area vs dernier commit
# (ce qui sera dans le prochain commit)
$ git diff --staged

# Comparer deux commits
$ git diff HEAD~1 HEAD

# Comparer un fichier spécifique
$ git diff README.md
```

Commande	Ce qu'elle compare
git diff	Working Directory ↔ Staging Area (non indexé)
git diff --staged	Staging Area ↔ Dernier commit (ce qui sera commité)
git diff HEAD	Working Directory ↔ Dernier commit (tout ce qui a changé)
git diff commit1 commit2	Deux commits quelconques de l'historique

RÉFLEXION : git diff vs git diff --staged

La distinction entre ces deux commandes résume parfaitement le concept des 3 zones :

• git diff vous montre ce que vous avez fait depuis votre dernier git add (modifications dans le Working Directory non encore indexées).

• git diff --staged vous montre ce qui va être enregistré dans votre prochain commit (contenu de la Staging Area par rapport au dernier commit).

Avant chaque commit, utilisez git diff --staged pour vérifier exactement ce que vous êtes sur le point d'enregistrer dans l'historique.

À TESTER — Mettez en pratique

1. Explorez l'historique avec git log et ses options :

```
$ git log
$ git log --oneline
$ git log --oneline --graph --all
```

2. Comparez les différentes zones avec git diff :

```
$ echo 'modification' >> app.txt
$ git diff                    ← WD vs Staging (non indexé)
$ git add app.txt
$ git diff                    ← plus rien (déjà indexé)
$ git diff --staged           ← Staging vs dernier commit
```

3. Utilisez la notation HEAD~ pour comparer :

```
$ git diff HEAD~1             ← qu'est-ce qui a changé sur le dernier
commit ?
$ git diff HEAD~2 HEAD        ← différence entre 2 commits
```

7. Ignorer des Fichiers : .gitignore

Garder votre dépôt propre et sécurisé

Dans tout projet, certains fichiers ne doivent jamais être versionnés : les fichiers de configuration contenant des mots de passe, les dossiers de dépendances générés automatiquement, les fichiers temporaires du système d'exploitation... Le fichier .gitignore est votre liste d'exclusions.

7.1 Créer et utiliser .gitignore

Le fichier .gitignore se place à la racine du projet (il peut aussi exister dans des sous-dossiers). Chaque ligne représente un pattern (motif) de fichiers à ignorer.

```
# .gitignore pour un projet Python
# Exemple de .gitignore complet pour un projet Python

# Fichiers d'environnement (SECRETS !)
.env
.env.local

# Dossier de dépendances Python
__pycache__/
*.pyc
*.pyo
venv/
.venv/

# Fichiers de logs
*.log
logs/

# Fichiers système
.DS_Store       # macOS
Thumbs.db       # Windows

# IDE
.vscode/
.idea/
```

7.2 Syntaxe des patterns

Pattern	Signification	Exemple d'utilisation

*.log	Ignore TOUS les fichiers se terminant par .log	error.log, debug.log
logs/	Ignore le dossier logs/ et tout son contenu	logs/app.log, logs/error.log
!important.log	Exception : NE PAS ignorer ce fichier (malgré *.log)	important.log sera suivi
*.log\n!important.log	Ignorer tous .log SAUF important.log	Combinaison règle + exception
/temp	Ignorer temp/ dans n'importe quel sous-dossier	src/temp/, app/temp/
doc/*.txt	Ignorer les .txt seulement dans doc/ (pas les sous-dossiers)	doc/notes.txt

L'EXCEPTION AVEC !

L'opérateur ! permet d'exclure certains fichiers d'une règle générale. L'ordre des règles est important : la dernière règle applicable l'emporte.

Exemple courant : ignorer tous les logs sauf le log critique :

```
*.log
!important.log
```

Si vous inversez l'ordre (!important.log puis *.log), la règle *.log écrasera l'exception et important.log sera quand même ignoré.

7.3 .gitignore sur des fichiers déjà suivis

Attention : .gitignore n'a aucun effet sur les fichiers déjà suivis par Git (déjà commités). Si vous ajoutez un fichier à .gitignore APRÈS l'avoir commité, Git continuera à le suivre.

PROBLÈME FRÉQUENT

Vous avez accidentellement commité node_modules/ ou un fichier .env. Ajouter ces chemins à .gitignore ne changera rien : Git les suit déjà.

LA SOLUTION : utiliser git rm --cached pour retirer le fichier du suivi de Git sans le supprimer de votre disque dur :

```
# Retirer des fichiers déjà suivis
# Retirer un fichier du suivi Git (sans le supprimer du disque)
$ git rm --cached .env

# Retirer un dossier entier du suivi
$ git rm -r --cached node_modules/

# Ensuite, commiter le retrait
$ git commit -m "Retirer .env et node_modules du suivi Git"

# Ajouter au .gitignore pour ne plus jamais les suivre
# (ajouter .env et node_modules/ dans .gitignore)
```

RÉFLEXION : que fait git rm --cached ?

La commande git rm --cached <fichier> retire le fichier de la Staging Area et du suivi de Git, mais NE supprime PAS le fichier sur votre disque dur. Le fichier reste présent dans votre Working Directory.

Sans l'option --cached, git rm supprimerait le fichier à la fois du suivi Git ET de votre disque dur (comme un rm classique). L'option --cached protège vos données.

À TESTER — Mettez en pratique

1. Créez un .gitignore et testez les patterns :

```
$ echo '*.log' > .gitignore
$ echo '*.tmp' >> .gitignore
$ touch debug.log erreur.tmp notes.txt
$ git status    ← .log et .tmp doivent être invisibles
```

2. Ajoutez une exception pour garder un seul fichier log :

```
$ echo '!important.log' >> .gitignore
$ touch important.log
$ git status    ← important.log doit apparaître comme Untracked
```

3. Testez git rm --cached sur un fichier déjà commité :

```
$ echo 'secret=password' > .env
$ git add .env && git commit -m "Ajout .env (erreur!)"
$ echo '.env' >> .gitignore
$ git status    ← .env est toujours suivi !
$ git rm --cached .env
$ git commit -m "Retirer .env du suivi Git"
$ git status    ← maintenant .env est ignoré
```

8. Annuler des Modifications

restore · revert · reset — Choisir la bonne commande

Git offre plusieurs façons d'annuler des modifications, chacune adaptée à une situation différente. La règle d'or : plus vous êtes avancé dans le flux (Working → Staging → Repository), plus l'annulation a d'impact potentiel.

```
Niveau d'annulation :

Working Directory   |   Staging Area    |   Repository (.git)
___________________ | _________________ | ___________________
git restore         | git restore       | git revert   (sûr)
<fichier>           | --staged          | git reset    (risqué)
(annule changements) | (désindexe)      | (réécrit historique)
```

Carte des commandes d'annulation

8.1 git restore : annuler dans le Working Directory et la Staging Area

git restore est la commande d'annulation sûre pour les modifications non encore commitées. Elle permet de revenir à l'état de la Staging Area (ou du dernier commit selon le contexte).

```
# git restore : les deux usages
# ANNULER les modifications d'un fichier dans le Working Directory
# (revenir à l'état de la Staging Area ou du dernier commit)
$ git restore README.md

# DÉSINDEXER un fichier (le retirer de la Staging Area)
# sans perdre les modifications dans le Working Directory
$ git restore --staged README.md

# Désindexer tous les fichiers
$ git restore --staged .
```

Commande	Action	Perd les modifications ?
git restore <fichier>	Annule les modifs du WD (revient au dernier git add ou commit)	**OUI** (WD modifié → perdu)
git restore --staged <fichier>	Retire le fichier de la Staging Area	**NON** (modifs restent dans WD)

ATTENTION : git restore est irréversible

git restore <fichier> (sans --staged) supprime DÉFINITIVEMENT les modifications non commitées du Working Directory. Ces modifications ne sont PAS dans l'historique Git et ne pourront donc pas être récupérées.

Vérifiez TOUJOURS avec git status et git diff avant d'utiliser git restore.

8.2 git revert : annuler un commit de façon sûre

git revert crée un nouveau commit qui effectue l'inverse exact d'un commit existant. L'historique original est préservé : vous pouvez voir qu'une modification a été faite puis annulée. C'est la méthode recommandée pour annuler des commits déjà partagés avec d'autres personnes.

```
# git revert en pratique
# Annuler le dernier commit (crée un nouveau commit d'annulation)
$ git revert HEAD

# Annuler un commit spécifique par son SHA-1
$ git revert a3f7c21

# Après : l'historique ressemble à :
# ... → commit-original → Revert "commit-original"
```

RÈGLE D'OR : préférer revert pour les commits partagés

Si vous avez déjà poussé vos commits sur un dépôt distant partagé avec d'autres développeurs, utilisez TOUJOURS git revert plutôt que git reset.

Pourquoi ? Parce que git revert ajoute de l'historique (un nouveau commit d'annulation) sans modifier l'historique existant. git reset, lui, réécrit l'historique, ce qui crée des conflits irréconciliables pour vos collègues qui ont déjà récupéré vos commits.

8.3 git reset : revenir dans le passé

git reset est une commande puissante mais délicate qui déplace le pointeur HEAD vers un commit antérieur, "effaçant" les commits intermédiaires de l'historique. Son comportement varie selon le mode utilisé.

```
# Les trois modes de git reset
# MODE SOFT : revient dans l'historique, garde les modifs dans la Staging
Area
$ git reset --soft HEAD~1

# MODE MIXED (défaut) : revient dans l'historique, garde les modifs dans
le WD
$ git reset HEAD~1        # équivalent à --mixed

# MODE HARD : revient dans l'historique et SUPPRIME toutes les
modifications
$ git reset --hard HEAD~1

# DANGER : reset --hard vers un commit spécifique (tout est perdu après
ce commit)
$ git reset --hard a3f7c21
```

Mode	HEAD	Staging Area	Working Directory	Usage recommandé
--soft	Déplacé	Conservée (commitée → staged)	Inchangé	Refaire le message de commit
--mixed	Déplacé	Vidée (commitée → WD)	Inchangé	Réorganiser les fichiers
--hard	Déplacé	**Effacée**	**Effacé**	Abandon total des changements

git reset --hard : avertissement

git reset --hard efface DÉFINITIVEMENT et IRRÉVERSIBLEMENT toutes les modifications non commitées et supprime les commits de l'historique local.

Utilisez-le uniquement quand vous êtes CERTAIN de vouloir perdre ces données et uniquement sur des commits qui n'ont PAS encore été partagés avec d'autres.

Si des collègues ont déjà récupéré les commits que vous réinitialisez, vous créerez des conflits graves dans le dépôt partagé.

À TESTER — Mettez en pratique

1. Testez git restore pour annuler des modifications non commitées :

```
$ echo 'ligne à supprimer' >> app.txt
$ git diff                    ← voir les modifications
$ git restore app.txt         ← annuler (IRRÉVERSIBLE !)
$ git diff                    ← plus aucune modification
```

2. Testez git revert pour annuler un commit :

```
$ echo 'commit à annuler' >> app.txt && git commit -am "Commit à
annuler"
$ git log --oneline           ← noter le SHA-1 du commit
$ git revert HEAD             ← crée un nouveau commit d'annulation
$ git log --oneline           ← l'historique montre les 2 commits
```

3. Testez les 3 modes de git reset (sur un dépôt de test) :

```
$ git reset --soft HEAD~1     ← revenir en arrière mais garder la
Staging Area
$ git status                  ← les fichiers sont staged
$ git reset HEAD~1            ← revenir en arrière, modifs dans WD
$ git status                  ← les fichiers sont Modified (non
staged)
$ git reset --hard HEAD~1     ← ⚠️  TOUT est effacé
```

4. Question : dans quel cas préférez-vous revert plutôt que reset ?

Récapitulatif des Commandes — Jour 1

Fiche mémo à conserver

Commande	Action	Moment d'utilisation
git init	Initialiser un nouveau dépôt Git	Une seule fois, au démarrage du projet
git config --global	Configurer l'identité (nom, email)	Une seule fois par machine
git status	Afficher l'état du dépôt	En permanence, avant chaque opération
git add <fichier>	Indexer des modifications (WD → Staging)	Avant chaque commit
git add .	Indexer toutes les modifications	Avec précaution (après config .gitignore)
git commit -m "..."	Enregistrer un commit avec message	Quand les modifs sont prêtes
git commit -am "..."	add + commit pour les fichiers suivis	Raccourci (fichiers déjà suivis uniquement)
git log	Afficher l'historique des commits	Pour naviguer dans le passé
git log --oneline	Historique compact (1 ligne par commit)	Vue rapide de l'historique
git diff	Comparer WD vs Staging Area	Voir les modifs non indexées
git diff --staged	Comparer Staging vs dernier commit	Vérifier avant de commiter
git restore <f>	Annuler les modifs dans le WD	Abandonner des modifications locales
git restore --staged <f>	Désindexer un fichier	Retirer un fichier de la Staging Area
git revert HEAD	Annuler un commit (nouveau commit)	Pour annuler des commits partagés
git reset --soft	Revenir en arrière (garde la Staging)	Corriger un message de commit
git reset --hard	Revenir en arrière (tout effacer)	⚠️ Abandon total, irreversible
git rm --cached <f>	Retirer du suivi sans supprimer du disque	Après oubli dans .gitignore